ECSE 597 Circuit Simulation and Modelling

Assignment 2

Due on Mar 17th, 2026

Chenyi Xu 260948311

1 Question 1

1.1 Q1.1

The function is coded using the formula below:

Backward Euler

$$\left(\mathbf{G}+\frac{\mathbf{C}}{h}\right)\mathbf{x}_{n+1}=\frac{\mathbf{C}}{h}\mathbf{x}_n+\mathbf{b}_{n+1}$$

Figure 1: Backward Euler Formula

Function for Backward Euler is shown below. Matrices G, C, and vector b are derived from the stamps. At each time step, the method solves the system using the equation mentioned above and records the output node voltage over time. The eval function in Matlab evaluate a string as a mathematical expression. The input source is provided as a string. Eval computes its value at the specified time t.

```
function [tpoints, r] = beuler(t1, t2, h, in, out)
global G
global C
global b
%t1 starting time
%t2 ending time
%h step size
%in: input source sin(2pi 1000t)
%out: output node
%tpoints: time points output was evaluated
%r: value of the response @tpoints
tpoints = t1:h:t2;
N = length(tpoints);
n = size(G,1);
x = zeros(n,N);                %initialize x
r = zeros(1, N);
r(1) = x(out, 1);              %initial value included
constant = G + C/h;
for a = 1 : N-1
    t = tpoints(a + 1);
    u = eval(in);
    right = b*u + (C/h)*x(:, a);
    x(:, a+1) = constant\right;
    r(a+1) = x(out, a+1);      %output
end                            %end for
end                            %end function
```

1.2 Q1.2

Specifying the input as required. The code used to find the transient response is shown below:

```
addpath('A1')
CR2 %find G, C, b
t1 = 0;
t2 = 7.5e-3;
h = 5e-6;
in = 'sin(2*pi*2000*t)';
out = 10;
[tpoints, r] = beuler(t1, t2, h, in, out);
```

1.3 Q1.3

The transient response plotted is shown below:

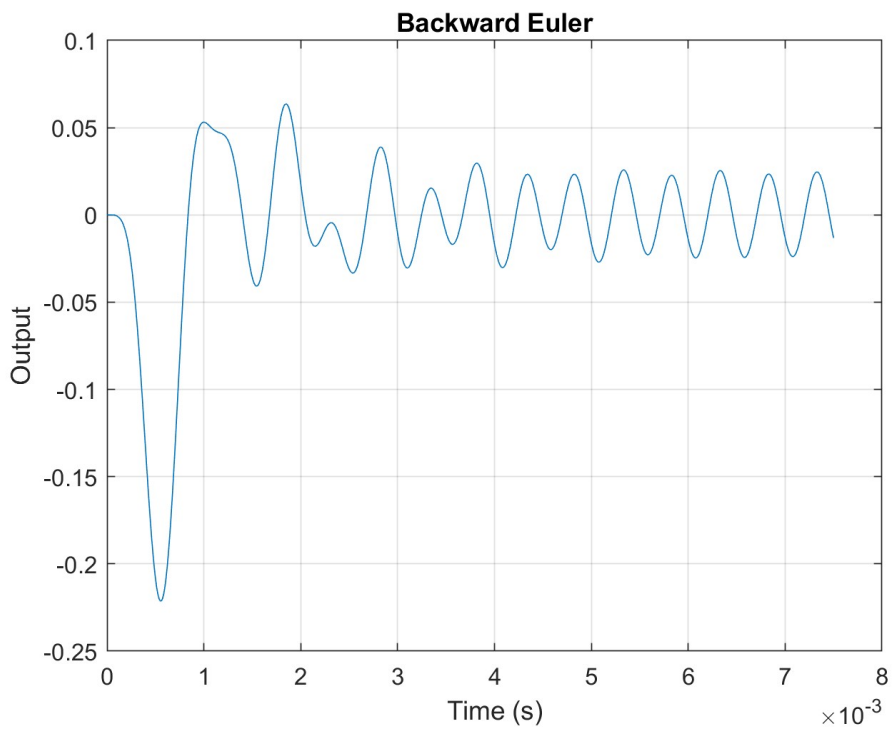

Figure 2: Transient Reponse Using Backward Euler

1.4 Q1.4

Using the fsolve function developed in Assignment 1, the frequency response is shown below. At 2kHZ, the magnitude is -32.1522dB.

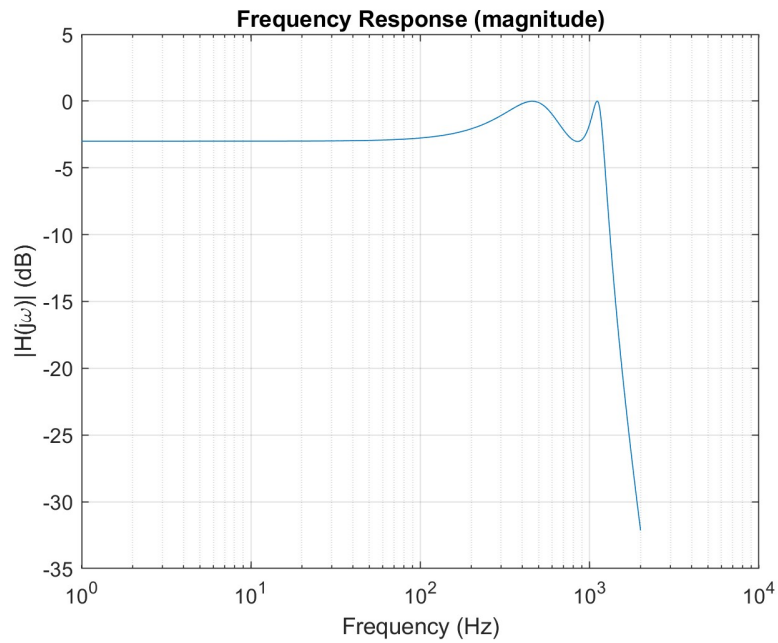

Figure 3: fsolve Magnitude

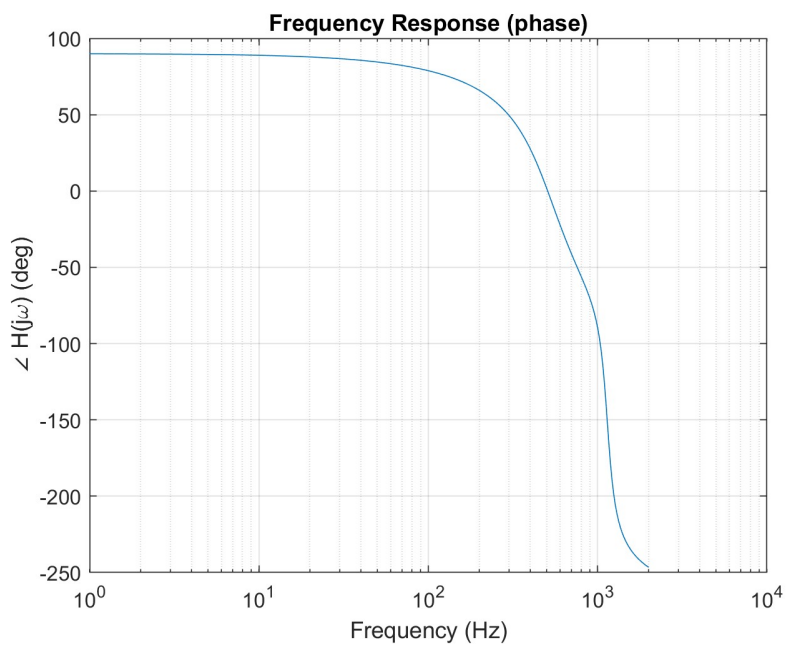

Figure 4: fsolve Phase

1.5 Q1.5

Convert the frequency response magnitude to linear magnitude:

$$|H(j\omega)| = 10^{\frac{-32.1522}{20}} \tag{1}$$

$$\approx 0.0247 \tag{2}$$

The transient response initially contains a natural response due to circuit dynamics and initial conditions. As time progress, the transient component disappears and the output convergers to a steady-state sinusoidal waveform with input frequency equals to 2kHz. The amplitude of this steady state sinusoidal corresponds to that of the transfer function at 2kHz obtained from frequency response which is approximately 0.0247. This confirms steady-state portion agrees with the frequency response prediction.

1.6 Q1.6

The step size is changed to 5e-5 and 7e-5 respectively to examine the effect of step sizes.

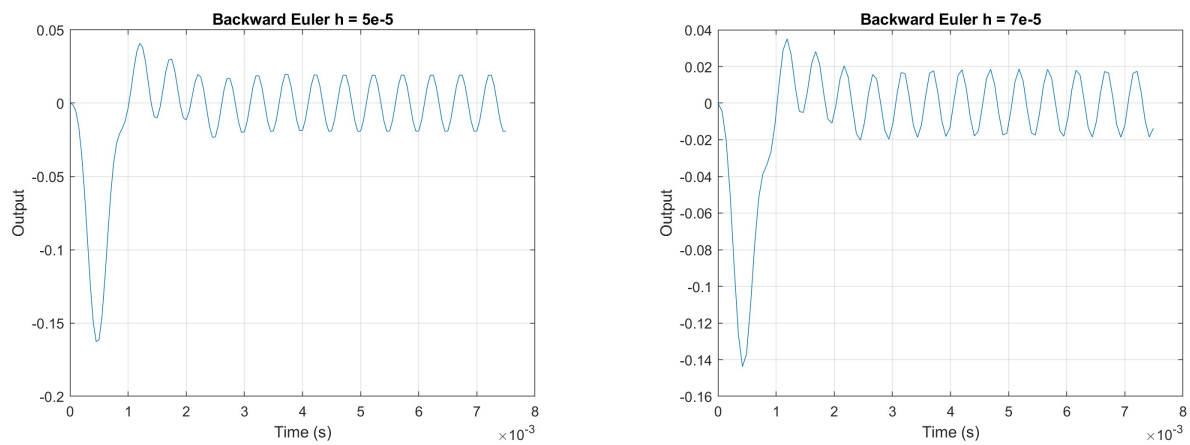

Figure 5: Step size of 5e-5 and 7e-5

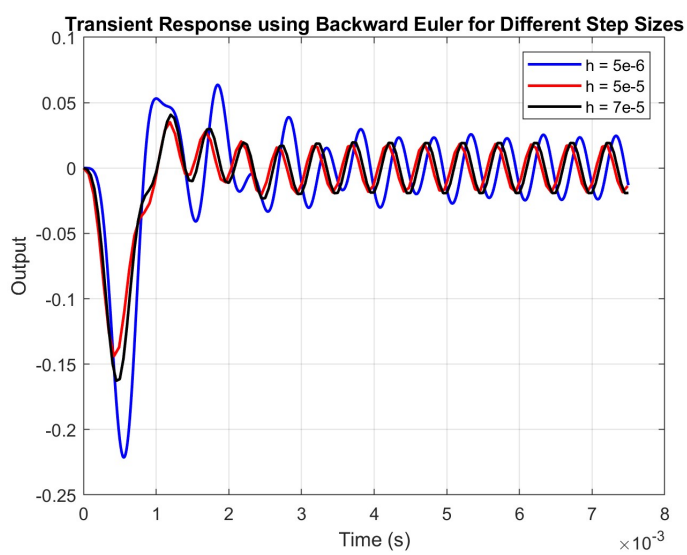

Figure 6: Overlap of Different Step sizes

Increase the step size reduces the numerical accuracy of the Backward Euler Integration. Larger step sizes introduce more dampings, which reduces the amplitude of the oscillations and leads to a less accurate transient response. Smaller step size outputs a more accurate representation of the circuit dynamics and a closer value towards frequency response output.

1.7 Q1.7

The function is coded using the formula below:

Trapezoidal Rule

$$\left(\mathbf{G}+\frac{2\mathbf{C}}{h}\right)\mathbf{x}_{n+1}=\left(\frac{2\mathbf{C}}{h}-\mathbf{G}\right)\mathbf{x}_n+\mathbf{b}_n+\mathbf{b}_{n+1}$$

Figure 7: Trapezoidal Formula

The function based on the trapezoidal rule is shown below. b is evaluated at both a and a + 1. At each time step, the method solves the system using the equation mentioned above and records the output node over time.

```
function [tpoints, r] = trapezoidal(t1, t2, h, in, out)
global G
global C
global b

tpoints = t1:h:t2;
N = length(tpoints);
n = size(G,1);

x = zeros(n,N);                 %initialize x
r = zeros(1, N);
r(1) = x(out, 1);               %initial value included

LHS_constant = G + 2*C/h;

RHS_constant = 2*C/h - G;

for a = 1 : N-1

    t = tpoints(a);             % b_n
    u = eval(in);

    t = tpoints(a+1);     % b_{n+1}
    u_next = eval(in);

    right =RHS_constant *x(:, a) + b*u + b*u_next;
    x(:, a+1) = LHS_constant\right;
    r(a+1) = x(out, a+1);       %output
end                             %end for function
end                             % end function
```

1.8 Q1.8

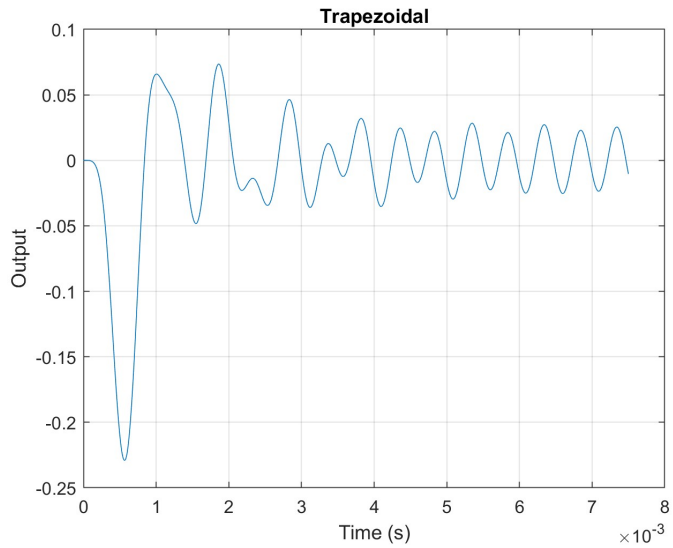

Figure 8: Trapezoidal output

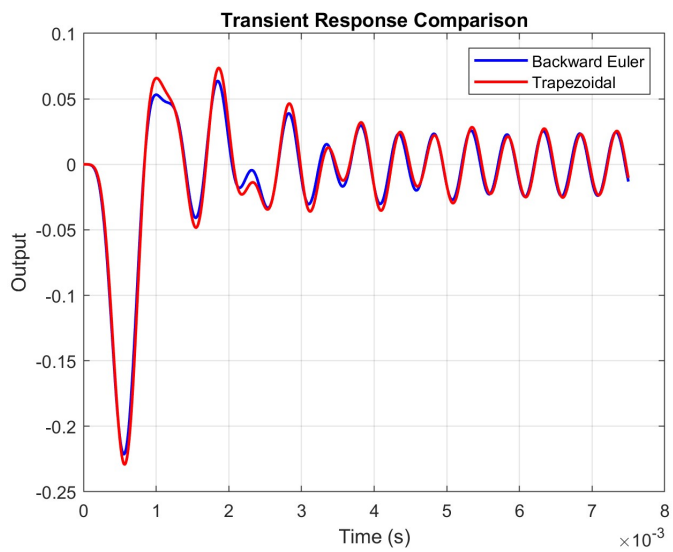

Figure 9: Trapezoidal Back Euler Comparison

Trapezoidal produces a less numerical damping response and therefore outputs a more accurate steady-state oscillation amplitude. Backward Euler is first-order accurate and numerically dissipative. It is like adding a small artificial resistance to the circuit. So the oscillations appear to be more damped than they should be. While the trapezoidal rule approximates the derivative using the average slope. It is second-order accurate and contains much less numerical damping. Oscillation amplitude stays closer to the true value.

2 Question 2

2.1 Q2.1

The code is created based on the formula:

$$CX_{n+1} = (C - hG)X_n + hb_n \tag{3}$$

At each time step, the input source is evaluated and the equation above is used to compute the next state. The output at the specified node is then stored to obtain the transient response over time.

```
function [tpoints, r] = feuler(t1, t2, h, in, out)
global G
global C
global b

tpoints = t1:h:t2;
N = length(tpoints);
n = size(G,1);

x = zeros(n,N);                %initialize x
r = zeros(1, N);
r(1) = x(out, 1);              %initial value included

LHS_constant = C ;
RHS_constant = C - h*G;

for a = 1 : N-1
    t = tpoints(a);
    u = eval(in);

    right = RHS_constant * x(:,a) + h*b*u;
    x(:, a+1) = LHS_constant\right;
    r(a+1) = x(out, a+1);      %output
end                            %end for

end                            %end function
```

2.2 Q2.2

Code for stamps in Figure 2 of the assignment

```
clear global G C b;
clc;
global G
global C
global b

G = zeros(4,4);
C = zeros(4,4);
b = zeros(4,1);
```

```
cur(1,0,1)

res(1,0,1)
cap(1,0,0.1e-9)

res(1,2,1)
cap(2,0,0.1e-9)

res(2,3,1)
cap(3,0,0.1e-9)

res(3,4,1)
cap(4,0,0.1e-9)
```

Backward Euler is performed with a constant step size of 0.001 ns using the code below.

```
circuit2
t1 = 0;
t2 = 15e-9;
h = 1e-12;
in = 'min(t/(50e-12),1)';
out = 4;
[tpoints, r] = beuler(t1, t2, h, in, out);
```

The response is shown below

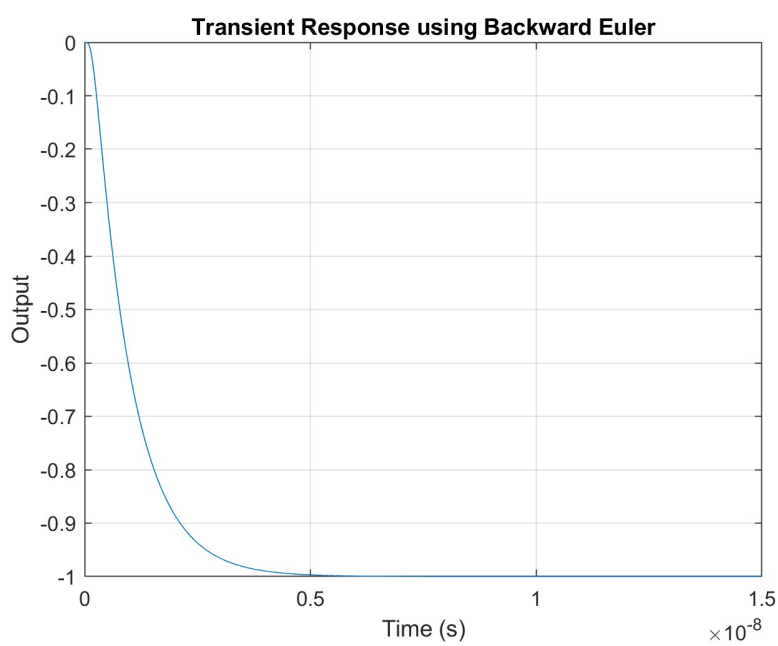

Figure 10: Backward Euler With a step size of 0.001ns

2.3 Q2.3

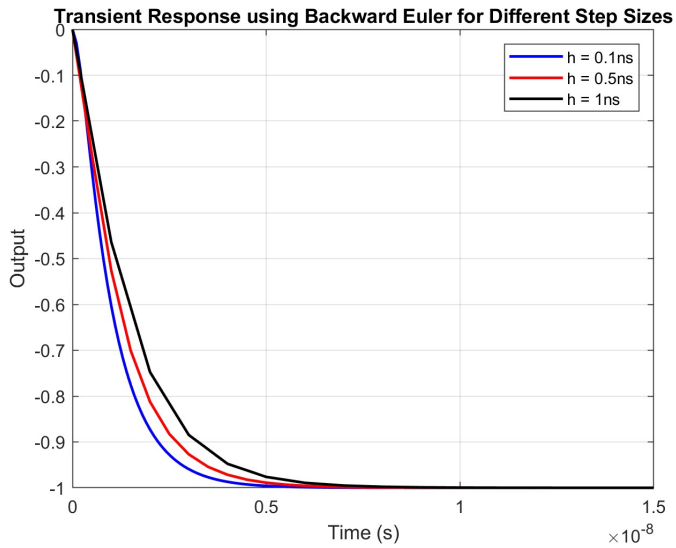

Figure 11: Backward Euler With Different Step Sizes

As the step size increases, the numerical solution obtained using the Backward Euler method becomes less accurate. The blue curve represents h = 0.1ns, which produces the transient behavior thats most close with the true behaviour of the circuit out of the three curves. When step sizes increase, the solution shows slower rises and deviates more from the true solution. This occurs because larger step sizes reduce the temporal resolution of the integration. However, even for large step sizes, backward euler is still stable and converges to the same steady-state value.

2.4 Q2.4

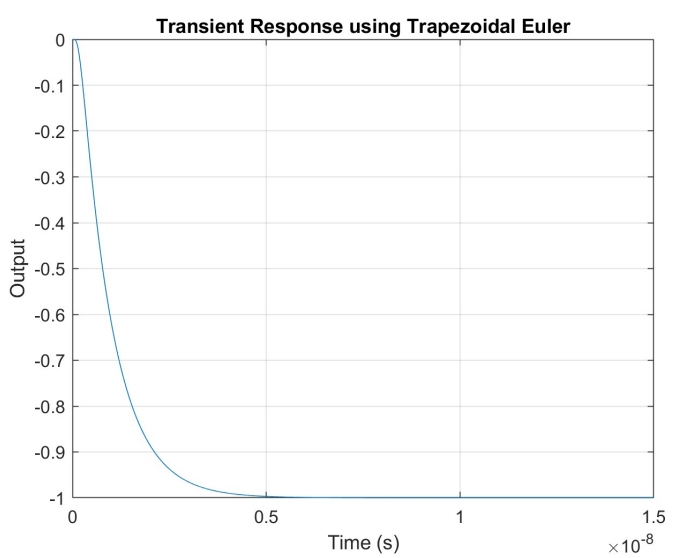

Figure 12: Trapezoidal With a step size of 0.001ns

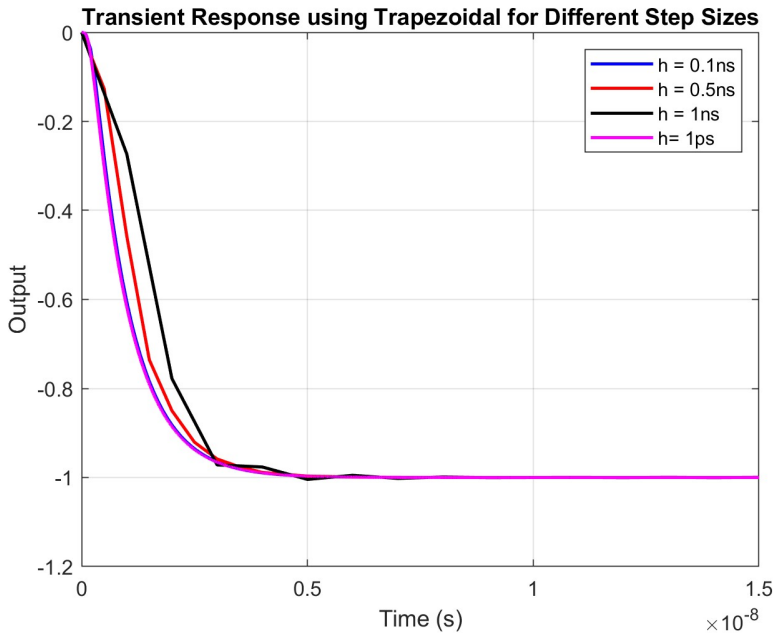

Figure 13: Trapezoidal With Different Step Sizes

As the step size increases the accuracy of trapezoidal integration decreases. For a smaller step size, the transient response is more smooth. When the step size increases to 0.5ns and 1ns, response begins to deviate from the true solution and small oscillations appear near steady-state value. The trapezoidal method has low numerical damping, which leads to the persistence of numerical errors when larger step sizes are used. However, they all achieves the same steady-state value despite the step sizes.

Compared with the Backward Euler, the trapezoidal generally provides higher accuracy for the same step size as it uses information from both the current and next time step. However, Backward Euler introduces stronger damping, which results in a smoother transient response. As a result, the Backward Euler solution is more stable for larger step sizes while the trapezoidal method may exhibits some oscillatory behaviour near steady-state. Trapezoidal is better for smaller step sizes.

2.5 Q2.5

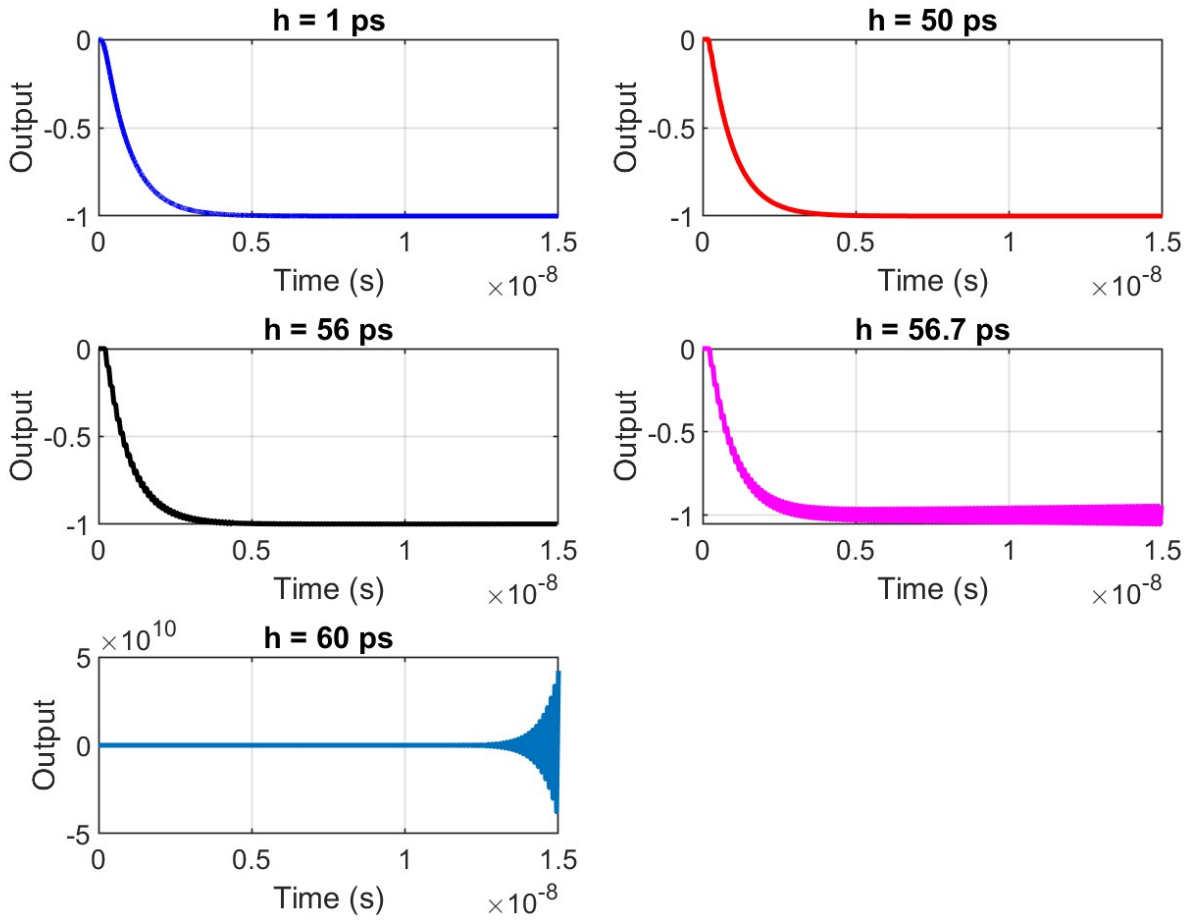

Figure 14: Forward Euler With Different Step Size

Forward Euler is very sensitive to the step size. For small step sizes such as 1 ps and 50 ps, the transient response is smooth. When the step size increases to 56 ps, small oscillations begin to appear, indicating an approach towards the stability limit. At 56.7 ps, the oscillations become more noticeable, and when the step size reaches 60 ps, the solution becomes unstable and diverges quickly. Forward Euler is conditionally stable. The time step needs to be smaller than the stability limit. If the step size exceeds this limit, the solution will diverge.

2.6 Q2.6

Poles of the system are the inverse of the eigenvalues of the matrix $-G^{-1}C$.
The code is shown below:

```
A = -inv(G)*C;
lambda = eig(A);
poles = 1 ./lambda
```

Ran the code, we obtained:

```
poles = 4×1
    -1.2061e+09
    -1.0000e+10
    -2.3473e+10
    -3.5321e+10
```

Figure 15: Poles

To find the range of step size for forward euler.

$$|1 + h\lambda| < 1 \tag{4}$$

$$h < \frac{2}{|\lambda|} \tag{5}$$

To find the eigenvalue, it is the pole value from ealier. Or we can find them from the state matrix. Either way we obtain the same value:
From the circuit equation we have:

$$C\dot{x} + Gx = b \tag{6}$$

$$\dot{x} = -C^{-1}Gx + C^{-1}b \tag{7}$$

We derive the state matrix to be

$$A = -C^{-1}G \tag{8}$$

Eigenvalues of A is the λ here.

```
A2 = -inv(C)*G;
lambda2 = eig(A2)
h_max2 = min(2./abs(lambda2))
```

Ran the code, we have

$$h_{max} = 56.624 \text{ ps} \tag{9}$$

Hence, the range is

$$0 < h < 56.624 \text{ ps} \tag{10}$$

The Forward Euler method is conditionally stable. The time step must respect the stability limit determined by the system poles. When the step size exceeds this limit, the numerical solution becomes unstable and

diverges rapidly. Even when stable, Forward Euler generally has a lower accuracy compared to other methods because it onluy uses information from the current time step.
The Backward Euler method is A-stable. It remains stable for any step size. However, it intrdouces numerical damping, which causes the transient response to appear smoother and slightly slower than true system behavior. This damping improves stability but can reduce accuracy.
The Trapezoidal method is also A-stable. It remains stable for any step sizes. It provides higher accuracy because it uses information from both the current and next time steps. However, it has less numerical damping, which may lead to small oscillations in the solution when larger step sizes are used.